

EXP1

```
!pip install scikit-fuzzy
```

```
import numpy as np
import skfuzzy as fuzz
from skfuzzy import control as ctrl
```

```
temperature = ctrl.Antecedent(np.arange(0, 101, 1), 'temperature')
fan_speed = ctrl.Consequent(np.arange(0, 101, 1), 'fan_speed')
```

```
temperature['low'] = fuzz.trimf(temperature.universe, [0, 0, 50])
temperature['medium'] = fuzz.trimf(temperature.universe, [0, 50, 100])
temperature['high'] = fuzz.trimf(temperature.universe, [50, 100, 100])
```

```
fan_speed['low'] = fuzz.trimf(fan_speed.universe, [0, 0, 50])
fan_speed['medium'] = fuzz.trimf(fan_speed.universe, [0, 50, 100])
fan_speed['high'] = fuzz.trimf(fan_speed.universe, [50, 100, 100])
```

```
rule1 = ctrl.Rule(temperature['low'], fan_speed['low'])
rule2 = ctrl.Rule(temperature['medium'], fan_speed['medium'])
rule3 = ctrl.Rule(temperature['high'], fan_speed['high'])
```

```
fan_ctrl = ctrl.ControlSystem([rule1, rule2, rule3])
fan_speed_ctrl = ctrl.ControlSystemSimulation(fan_ctrl)
```

```
temperature_value = 75
```

```
fan_speed_ctrl.input['temperature'] = temperature_value
fan_speed_ctrl.compute()
```

```
print("Fan speed:", fan_speed_ctrl.output['fan_speed'])
```

```
temperature.view()
fan_speed.view()
fan_speed.view(sim=fan_speed_ctrl)
```

EXP2

```
import numpy as np
```

```
X = np.array([[2,3], [3,2], [1,1], [5,7], [6,8], [7,6]])
```

```
y = np.array([0,0,0,1,1,1])
```

```
w = np.zeros(2)
```

```
b = 0
```

```
for _ in range(100):
```

```
    for xi, yi in zip(X, y):
```

```
        output = np.dot(w, xi) + b
```

```
        pred = 1 if output > 0 else 0
```

```
        error = yi - pred
```

```
        w += 0.1 * error * xi
```

```
        b += 0.1 * error
```

```
test_data = np.array([[4,5], [2,2]])
```

```
for data in test_data:
```

```
    output = np.dot(w, data) + b
```

```
    pred = 1 if output > 0 else 0
```

```
if pred == 0:
```

```
    print(f"Data {data} belongs to class 0")
```

```
else:
```

```
    print(f"Data {data} belongs to class 1")
```

EXP3

```
import numpy as np
```

```
def sigmoid(x):  
    return 1 / (1 + np.exp(-x))
```

```
X = np.array([[0,0],  
              [0,1],  
              [1,0],  
              [1,1]])
```

```
y = np.array([[0],  
              [1],  
              [1],  
              [0]])
```

```
w1 = np.random.rand(2,2)  
w2 = np.random.rand(2,1)
```

```
for _ in range(10000):
```

```
    h = sigmoid(np.dot(X, w1))  
    o = sigmoid(np.dot(h, w2))
```

```
    d2 = (y - o) * o * (1 - o)  
    d1 = d2.dot(w2.T) * h * (1 - h)
```

```
    w2 += h.T.dot(d2) * 0.1  
    w1 += X.T.dot(d1) * 0.1
```

```
    for i in X:  
        h = sigmoid(np.dot(i, w1))  
        o = sigmoid(np.dot(h, w2))
```

```
    print(f"Input: {i} Predicted Output: {o[0]}")
```

EXP4

```
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(42)
data = np.random.rand(100, 2)

grid = (10, 10)
w = np.random.rand(grid[0], grid[1], 2)
for _ in range(500):

    for x in data:

        d = np.linalg.norm(w - x, axis=2)

        bmu = np.unravel_index(np.argmin(d), d.shape)

        for i in range(grid[0]):
            for j in range(grid[1]):
                dist = np.linalg.norm([i - bmu[0], j - bmu[1]])
                influence = np.exp(-dist**2 / 10)
                w[i, j] += 0.2 * influence * (x - w[i, j])
            cluster = np.zeros(grid)
            for i in range(grid[0]):
                for j in range(grid[1]):

                    d = np.linalg.norm(data - w[i, j], axis=1)

                    cluster[i, j] = np.argmin(d)

        plt.pcolormesh(cluster, cmap='viridis')
        plt.colorbar()

        plt.scatter(data[:,0] * 10, data[:,1] * 10, c='red')

        plt.title("Self-Organizing Map Clustering")

        plt.show()
```

EXP5

```
import random
```

```
def f(x):  
    return -x**2 + 6*x + 9
```

```
# Initial population  
pop = [random.uniform(-10, 10) for _ in range(50)]
```

```
for gen in range(50):
```

```
    new_pop = []
```

```
    for _ in range(50):
```

```
        p1 = random.choice(pop)  
        p2 = random.choice(pop)
```

```
# Crossover  
        c = (p1 + p2) / 2
```

```
# Mutation  
        if random.random() < 0.01:  
            c += random.uniform(-1, 1)
```

```
    new_pop.append(c)
```

```
    pop = new_pop
```

```
    best = max(pop, key=f)
```

```
    print(f"Generation {gen+1}: Best individual - {best}, Fitness - {f(best)}")
```

```
    best = max(pop, key=f)
```

```
    print(f"Best solution found: {best}, Fitness: {f(best)}")
```

EXP6

```
import random
import math
def f(x, y):
    return math.sin(x) + math.sin(y)
# Initial population
pop = [
    (
        random.uniform(-2 * math.pi, 2 * math.pi),
        random.uniform(-2 * math.pi, 2 * math.pi)
    )
    for _ in range(100)
]
for gen in range(20):
    new_pop = []

    for _ in range(50):

        p1 = random.choice(pop)
        p2 = random.choice(pop)

        # Crossover
        c = (p1[0], p2[1])

        # Mutation
        if random.random() < 0.01:
            c = (
                c[0] + random.uniform(-0.1, 0.1),
                c[1] + random.uniform(-0.1, 0.1)
            )
        new_pop.append(c)
    pop = new_pop
    best = max(pop, key=lambda i: f(i[0], i[1]))
    print(f"Generation {gen+1}: Best individual - {best}, Fitness - {f(best[0], best[1])}")
    best = max(pop, key=lambda i: f(i[0], i[1]))
    print(f"Best solution found: {best}, Fitness: {f(best[0], best[1])}")
```